

# An Analysis of Attention mechanism neural network

Attention mechanism neural network

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

## Attention mechanism neural network -- Part 1

where  $Q$ ,  $K$ ,  $V$  are respectively the query, key, value matrices, and  $d_k$  is the dimension of the values. Since the model relies on Query (Q), Key (K), and Value (V) matrices that come from the same source (i.e., the input sequence or context window), this eliminates the need for RNNs, completely ensuring parallelizability for the architecture. This differs from the original form of the Attention mechanism introduced in 2014. Additionally, the paper also discusses the use of an additional scaling factor that was found to be most effective with respect to the dimension of the key vectors (represented as  $d_k$  and initially set to 64 within the paper) in the manner shown above. In the specific context of translation, which the paper focused on, the Query and Key matrices are usually represented in embeddings corresponding to the source language, while the Value matrix corresponds to the target language.

## Attention mechanism neural network -- Part 2

wherein  $pos$ ,  $i$ ,  $d_{model}$  correspond to the position of the word, the current dimension index, and the dimension of the model, respectively. The sine function is used for even indices of the embedding while the cosine function is used for odd indices. The resultant  $PE$  embedding is then added to the word at that corresponding position with respect to the current context window. The paper specifically comments on why this method was chosen describing: "We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training."

## Attention mechanism neural network -- Part 3

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{(2i+1)/d_{model}}}\right)$$

## Attention mechanism neural network -- Part 4

AI boom era As early as spring 2017, even before the "Attention is all you need" preprint was published, one of the co-authors applied the "decoder-only" variation of the architecture to

generate fictitious Wikipedia articles. Transformer architecture is now used alongside many generative models that contribute to the ongoing AI boom. The "reference implementation" of the original Transformer was written in a TensorFlow library. In language modelling, ELMo (2018) was a bi-directional LSTM that produces contextualized word embeddings, improving upon the line of research from bag of words and word2vec. It was followed by BERT (2018), an encoder-only transformer model. In October 2019, Google started using BERT to process search queries. In 2020, Google Translate replaced the previous RNN-encoder-RNN-decoder model by a transformer-encoder-RNN-decoder model. Starting in 2018, the OpenAI GPT series of decoder-only transformers became state of the art in natural language generation. At the end of 2022, ChatGPT, a chatbot based on a fine-tuned variant of GPT-3.5, became unexpectedly popular, triggering a boom around large language models. Transformers have been applied in modalities beyond text. Four days after the publication of "Attention is All You Need", a multimodal transformer architecture, MultiModel, was published by most authors of that paper. Other examples include the vision transformer, speech recognition, robotics, and multimodal. The vision transformer, in turn, stimulated new developments in convolutional neural networks. Image and video generators like DALL-E (2021), Stable Diffusion 3 (2024), and Sora (2024), use transformers to analyse input data (like text prompts) by breaking it down into "tokens" and then calculating the relevance between each token using self-attention, which helps the model understand the context and relationships within the data.

## Attention mechanism neural network -- Part 5

Multi-head attention In the self-attention mechanism, queries (Q), keys (K), and values (V) are dynamically generated for each input sequence (typically limited by the size of the context window), allowing the model to focus on different parts of the input sequence at different steps. Multi-head attention enhances this process by introducing multiple parallel attention heads. Each attention head learns different linear projections of the Q, K, and V matrices. This allows the model to capture different aspects of the relationships between words in the sequence simultaneously, rather than focusing on a single aspect. By doing this, multi-head attention ensures that the input embeddings are updated from a more varied and diverse set of perspectives. After the attention outputs from all heads are calculated, they are concatenated and passed through a final linear transformation to generate the output.

## Attention mechanism neural network -- Part 6

# An Analysis of Attention mechanism neural network

*Attention mechanism neural network*

**Predecessors** For many years, sequence modelling and generation was done by using plain recurrent neural networks (RNNs). A well-cited early example was the Elman network (1990). In theory, the information from one token can propagate arbitrarily far down the sequence, but in practice the vanishing-gradient problem leaves the model's state at the end of a long sentence without precise, extractable information about preceding tokens. A key breakthrough was LSTM (originally described in a 1995 technical report and formally published in 1997), an RNN that introduced gating mechanisms to mitigate the vanishing gradient problem, allowing efficient learning of long-sequence modelling. One key architectural element was the use of multiplicative gating units, in which the outputs of some neurons modulate the outputs of others. These multiplicative units are conceptually distinct from the additive attention mechanism later introduced for sequence-to-sequence models. Neural networks using multiplicative units were later called sigma-pi networks or higher-order networks. LSTM became the standard architecture for long sequence modelling until the 2017 publication of transformers. However, LSTM still used sequential processing, like most other RNNs. Specifically, RNNs operate one token at a time from first to last; they cannot operate in parallel over all tokens in a sequence. Modern transformers overcome this problem, but unlike RNNs, they require computation time that is quadratic in the size of the context window. The linearly scaling fast weight controller (1992) learns to compute a weight matrix for further processing depending on the input. One of its two networks has "fast weights" or "dynamic links" (1981). A slow neural network learns by gradient descent to generate keys and values for computing the weight changes of the fast neural network which computes answers to queries. This was later shown to be equivalent to the unnormalized linear transformer.